Contour Software — Coding Interview

Java • Python • C++ • DSA • OOP • SQL Scenarios • LeetCode Style • LinkedList • Priority Queue • Trees

Muhammad Faizan | Trainee Software Developer | Complete Coding Prep

SECTION 1A — OOP IMPLEMENTATION: JAVA & PYTHON

1. Complete OOP Class — Bank Account

Java

```java
public class BankAccount {
  private String owner;
  private double balance;

  // Constructor
  public BankAccount(String owner, double initial) {
    this.owner  = owner;
    this.balance = initial;
  }

  // Deposit
  public void deposit(double amount) {
    if (amount > 0) balance += amount;
  }

  // Withdraw with validation
  public boolean withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
      balance -= amount;
      return true;
    }
    return false;
  }

  public double getBalance() { return balance; }
  public String getOwner()   { return owner; }

  @Override
  public String toString() {
    return owner + ": PKR " + balance;
  }

  public static void main(String[] args) {
    BankAccount acc = new BankAccount("Faizan", 50000);
    acc.deposit(10000);
    acc.withdraw(5000);
    System.out.println(acc); // Faizan: PKR 55000.0
  }
}
```

Python

```python
class BankAccount:
  def __init__(self, owner, initial):
    self.__owner   = owner     # private
    self.__balance = initial

  def deposit(self, amount):
    if amount > 0:
      self.__balance += amount

  def withdraw(self, amount):
    if 0 < amount <= self.__balance:
      self.__balance -= amount
      return True
    return False

  def get_balance(self): return self.__balance
  def get_owner(self):   return self.__owner

  def __str__(self):
    return f"{self.__owner}: PKR {self.__balance}"

acc = BankAccount("Faizan", 50000)
acc.deposit(10000)
acc.withdraw(5000)
print(acc)  # Faizan: PKR 55000
```

2. Inheritance — Shape Hierarchy

Java

Python

```java
abstract class Shape {
  String color;
  Shape(String color) { this.color = color; }

  abstract double area();  // must implement

  void display() {
    System.out.println(color + " shape, area=" + area());
  }
}

class Circle extends Shape {
  double radius;
  Circle(String color, double r) {
    super(color);
    this.radius = r;
  }
  @Override
  public double area() {
    return Math.PI * radius * radius;
  }
}

class Rectangle extends Shape {
  double w, h;
  Rectangle(String color, double w, double h) {
    super(color);
    this.w = w; this.h = h;
  }
  @Override
  public double area() { return w * h; }
}

// Main
Shape[] shapes = {
  new Circle("Red", 5),
  new Rectangle("Blue", 4, 6)
};
for (Shape s : shapes) s.display();
// Red shape, area=78.53...
// Blue shape, area=24.0
```

```python
from abc import ABC, abstractmethod
import math

class Shape(ABC):
  def __init__(self, color):
    self.color = color

  @abstractmethod
  def area(self): pass

  def display(self):
    print(f"{self.color} area={self.area():.2f}")

class Circle(Shape):
  def __init__(self, color, radius):
    super().__init__(color)
    self.radius = radius

  def area(self):
    return math.pi * self.radius ** 2

class Rectangle(Shape):
  def __init__(self, color, w, h):
    super().__init__(color)
    self.w = w; self.h = h

  def area(self): return self.w * self.h

shapes = [Circle("Red",5), Rectangle("Blue",4,6)]
for s in shapes:
  s.display()
```

3. Interface / Multiple Behaviour — Flyable & Swimmable

Java — Interface

```java
interface Flyable {
  void fly();
}
interface Swimmable {
  void swim();
}

class Duck implements Flyable, Swimmable {
  String name;
  Duck(String name) { this.name = name; }

  @Override public void fly() {
    System.out.println(name + " is flying");
  }
  @Override public void swim() {
    System.out.println(name + " is swimming");
  }
}

Duck d = new Duck("Donald");
d.fly();   // Donald is flying
d.swim();  // Donald is swimming
```

Python — Duck Typing

```python
class Flyable:
  def fly(self):
    raise NotImplementedError

class Swimmable:
  def swim(self):
    raise NotImplementedError

class Duck(Flyable, Swimmable):
  def __init__(self, name):
    self.name = name

  def fly(self):
    print(f"{self.name} is flying")

  def swim(self):
    print(f"{self.name} is swimming")

d = Duck("Donald")
d.fly()   # Donald is flying
d.swim()  # Donald is swimming
```

SECTION 1B — DSA: ARRAYS & STRINGS (LeetCode Style)

Problem 1: Reverse an Array | **Difficulty:** Easy | **Tags:** Array, Two Pointers

Given an array, reverse it in-place without using extra space.

Java Solution:

```
void reverseArray(int[] arr) {
  int left = 0, right = arr.length - 1;
  while (left < right) {
    int temp  = arr[left];
    arr[left] = arr[right];
    arr[right]= temp;
    left++; right--;
  }
}
// Input:  [1, 2, 3, 4, 5]
// Output: [5, 4, 3, 2, 1]
```

Python Solution:

```
def reverse_array(arr):
  left, right = 0, len(arr) - 1
  while left < right:
    arr[left], arr[right] = arr[right], arr[left]
    left += 1
    right -= 1
  return arr
# Input:  [1, 2, 3, 4, 5]
# Output: [5, 4, 3, 2, 1]
```

✓ Two pointer technique. O(n) time, O(1) space. Swap elements from both ends moving inward.

Problem 2: Check Palindrome String | **Difficulty:** Easy | **Tags:** String, Two Pointers

Given a string, return true if it reads the same forwards and backwards.

Java Solution:

```
boolean isPalindrome(String s) {
  s = s.toLowerCase().replaceAll("[^a-z0-9]","");
  int l = 0, r = s.length()-1;
  while (l < r) {
    if (s.charAt(l) != s.charAt(r)) return false;
    l++; r--;
  }
  return true;
}
// "racecar" -> true
// "hello"   -> false
```

Python Solution:

```
def is_palindrome(s):
  s = ''.join(c for c in s.lower() if c.isalnum())
  return s == s[::-1]
# "racecar" -> True
# "hello"   -> False
```

✓ Clean string first (lowercase, remove non-alphanumeric). Compare from both ends. O(n) time.

Problem 3: Two Sum | **Difficulty:** Easy | **Tags:** Array, HashMap

Given array and target, return indices of two numbers that add up to target.

Java Solution:

```
int[] twoSum(int[] nums, int target) {
  HashMap map = new HashMap<>();
  for (int i = 0; i < nums.length; i++) {
    int complement = target - nums[i];
    if (map.containsKey(complement))
      return new int[]{map.get(complement), i};
    map.put(nums[i], i);
  }
  return new int[]{};
}
// Input:  nums=[2,7,11,15], target=9
// Output: [0, 1]
```

Python Solution:

```
def two_sum(nums, target):
  seen = {}
  for i, num in enumerate(nums):
    complement = target - num
    if complement in seen:
      return [seen[complement], i]
    seen[num] = i
  return []
# Input:  [2,7,11,15], target=9
# Output: [0, 1]
```

✓ HashMap stores value->index. For each number check if complement exists. O(n) time, O(n) space.

Problem 4: Find Maximum in Array | **Difficulty:** Easy | **Tags:** Array

Find the maximum value in an array without using built-in max function.

Java Solution:

```
int findMax(int[] arr) {
  int max = arr[0];
  for (int i = 1; i < arr.length; i++) {
    if (arr[i] > max) max = arr[i];
  }
  return max;
}
// Input:  [3, 1, 9, 2, 7]
// Output: 9
```

Python Solution:

```
def find_max(arr):
  max_val = arr[0]
  for num in arr[1:]:
    if num > max_val:
      max_val = num
  return max_val
```

✓ Initialize max as first element. Iterate and update if current > max. O(n) time, O(1) space.

Problem 5: Remove Duplicates from Array | **Difficulty:** Easy | **Tags:** Array, HashSet

Return array with duplicates removed, maintaining original order.

Java Solution:

```
int[] removeDuplicates(int[] arr) {
  LinkedHashSet set = new LinkedHashSet<>();
  for (int x : arr) set.add(x);
  int[] result = new int[set.size()];
  int i = 0;
  for (int x : set) result[i++] = x;
  return result;
}
// Input:  [1,2,2,3,3,3,4]
// Output: [1,2,3,4]
```

Python Solution:

```
def remove_duplicates(arr):
  seen = set()
  result = []
  for x in arr:
    if x not in seen:
      result.append(x)
      seen.add(x)
  return result
# [1,2,2,3,3,3,4] -> [1,2,3,4]
```

✓ LinkedHashSet preserves insertion order and removes duplicates automatically. O(n) time and space.

Problem 6: Count Frequency of Elements | **Difficulty:** Easy | **Tags:** Array, HashMap

Count how many times each element appears in an array.

Java Solution:

```
HashMap countFreq(int[] arr) {
  HashMap freq = new HashMap<>();
  for (int x : arr) {
    freq.put(x, freq.getOrDefault(x, 0) + 1);
  }
  return freq;
}
// Input:  [1,2,2,3,3,3]
// Output: {1:1, 2:2, 3:3}
```

Python Solution:

```
from collections import Counter
def count_freq(arr):
  return Counter(arr)
  # OR manually:
  # freq = {}
  # for x in arr:
  #   freq[x] = freq.get(x, 0) + 1
  # return freq
```

✓ getOrDefault(key, 0) returns 0 if key not in map. Python Counter does this automatically.

SECTION 1C — LINKED LIST (Complete)

LinkedList Node + Basic Operations

Java — Full LinkedList Implementation

Reverse a Linked List — Iterative

```
class Node {
  int data;
  Node next;
  Node(int data) {
    this.data = data;
    this.next = null;
  }
}

class LinkedList {
  Node head;

  // Add to end
  void append(int data) {
    Node newNode = new Node(data);
    if (head == null) { head = newNode; return; }
    Node curr = head;
    while (curr.next != null) curr = curr.next;
    curr.next = newNode;
  }

  // Add to front
  void prepend(int data) {
    Node newNode = new Node(data);
    newNode.next = head;
    head = newNode;
  }

  // Delete by value
  void delete(int data) {
    if (head == null) return;
    if (head.data == data) { head = head.next; return; }
    Node curr = head;
    while (curr.next != null) {
      if (curr.next.data == data) {
        curr.next = curr.next.next; return;
      }
      curr = curr.next;
    }
  }

  // Print all
  void print() {
    Node curr = head;
    while (curr != null) {
      System.out.print(curr.data + " -> ");
      curr = curr.next;
    }
    System.out.println("null");
  }
}
```

```
// Reverse LinkedList in-place
Node reverse(Node head) {
  Node prev = null;
  Node curr = head;
  while (curr != null) {
    Node next  = curr.next; // save next
    curr.next  = prev;      // reverse link
    prev = curr;            // move prev
    curr = next;            // move curr
  }
  return prev; // new head
}
// 1->2->3->4->null
// becomes
// 4->3->2->1->null
```

Python — LinkedList

```
class Node:
  def __init__(self, data):
    self.data = data
    self.next = None

class LinkedList:
  def __init__(self):
    self.head = None

  def append(self, data):
    new_node = Node(data)
    if not self.head:
      self.head = new_node; return
    curr = self.head
    while curr.next:
      curr = curr.next
    curr.next = new_node

  def reverse(self):
    prev, curr = None, self.head
    while curr:
      nxt = curr.next
      curr.next = prev
      prev = curr
      curr = nxt
    self.head = prev
```

Problem 7: Detect Cycle in LinkedList | **Difficulty:** Medium | **Tags:** LinkedList, Floyd's Algorithm

Detect if a linked list has a cycle (loop) using O(1) space.

Java Solution:

```
boolean hasCycle(Node head) {
  Node slow = head;
  Node fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next;        // move 1 step
    fast = fast.next.next;   // move 2 steps
    if (slow == fast) return true; // cycle!
  }
  return false;
}
```

Python Solution:

```
def has_cycle(head):
  slow = fast = head
  while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
    if slow == fast:
      return True
  return False
```

✓ Floyd's Tortoise and Hare. Slow moves 1 step, fast moves 2. If cycle exists they'll meet. O(n) time, O(1) space.

Problem 8: Find Middle of LinkedList | **Difficulty:** Easy | **Tags:** LinkedList, Two Pointers

Find the middle node of a linked list in one pass.

Java Solution:

```
Node findMiddle(Node head) {
  Node slow = head;
  Node fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
  }
  return slow; // slow is at middle
}
// 1->2->3->4->5 -> returns node 3
```

Python Solution:

```
def find_middle(head):
  slow = fast = head
  while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
  return slow
```

✓ When fast reaches end, slow is at middle. Fast moves 2x speed of slow. O(n) time, O(1) space.

SECTION 1D — STACK, QUEUE & PRIORITY QUEUE

Stack — Balanced Parentheses

Java

```
boolean isBalanced(String s) {
  Stack stack = new Stack<>();
  for (char c : s.toCharArray()) {
    if (c=='(' || c=='[' || c=='{') {
      stack.push(c);
    } else {
      if (stack.isEmpty()) return false;
      char top = stack.pop();
      if (c==')' && top!='(') return false;
      if (c==']' && top!='[') return false;
      if (c=='}' && top!='{') return false;
    }
  }
  return stack.isEmpty();
}
// "({[]})" -> true
// "([)]"   -> false
```

Python

```
def is_balanced(s):
  stack = []
  mapping = {')':'(', ']':'[', '}':'{'}
  for c in s:
    if c in '([{':
      stack.append(c)
    elif c in mapping:
      if not stack or stack[-1] != mapping[c]:
        return False
      stack.pop()
  return len(stack) == 0
# "({[]})" -> True
# "([)]"   -> False
```

Priority Queue — Elevator Simulation (Your Scenario)

Scenario: Elevator goes UP and DOWN. Requests have priorities. Process highest priority floor first. This is a classic Priority Queue problem.

Java — Priority Queue (Max Heap)

Python — heapq

```java
import java.util.PriorityQueue;
import java.util.Collections;

// Elevator with Priority Queue
// Higher floor number = higher priority (going up)
class ElevatorRequest implements Comparable {
  int floor;
  int priority;  // 1=normal, 2=emergency

  ElevatorRequest(int floor, int priority) {
    this.floor    = floor;
    this.priority = priority;
  }

  @Override
  public int compareTo(ElevatorRequest other) {
    // Higher priority first
    return other.priority - this.priority;
  }
}

PriorityQueue pq =
    new PriorityQueue<>();

pq.offer(new ElevatorRequest(3, 1)); // floor 3, normal
pq.offer(new ElevatorRequest(7, 2)); // floor 7, emergenc
y
pq.offer(new ElevatorRequest(1, 1)); // floor 1, normal

while (!pq.isEmpty()) {
  ElevatorRequest req = pq.poll();
  System.out.println("Go to floor: " + req.floor);
}
// Go to floor: 7 (emergency first)
// Go to floor: 3
// Go to floor: 1
```

```python
import heapq

# Python heapq is min-heap by default
# Use negative priority for max-heap behavior
class Elevator:
  def __init__(self):
    self.requests = []  # min-heap
    self.count = 0      # tiebreaker

  def add_request(self, floor, priority):
    # Negate priority for max-heap
    heapq.heappush(
      self.requests,
      (-priority, self.count, floor)
    )
    self.count += 1

  def process_next(self):
    if self.requests:
      _, _, floor = heapq.heappop(self.requests)
      return floor
    return None

elev = Elevator()
elev.add_request(3, 1)  # normal
elev.add_request(7, 2)  # emergency
elev.add_request(1, 1)  # normal

while True:
  floor = elev.process_next()
  if floor is None: break
  print(f"Go to floor: {floor}")
# Go to floor: 7
# Go to floor: 3 (or 1)
# Go to floor: 1 (or 3)
```

Problem 9: K Largest Elements | **Difficulty:** Medium | **Tags:** Array, Heap, Priority Queue

Find K largest elements from an array.

Java Solution:

```java
int[] kLargest(int[] arr, int k) {
  // Min-heap of size k
  PriorityQueue pq = new PriorityQueue<>();
  for (int num : arr) {
    pq.offer(num);
    if (pq.size() > k) pq.poll(); // remove smallest
  }
  int[] result = new int[k];
  int i = 0;
  while (!pq.isEmpty()) result[i++] = pq.poll();
  return result;
}
// Input: [3,2,1,5,6,4], k=2
// Output: [5,6]
```

Python Solution:

```python
import heapq
def k_largest(arr, k):
  return heapq.nlargest(k, arr)
  # Manual: use min heap of size k
  # heap = []
  # for num in arr:
  #   heapq.heappush(heap, num)
  #   if len(heap) > k:
  #     heapq.heappop(heap)
  # return heap
```

✓ Maintain min-heap of size k. If heap > k, remove smallest. Remaining k elements are the largest.

SECTION 1E — TREES & BINARY SEARCH TREE

Java — BST Implementation

```
class TreeNode {
  int val;
  TreeNode left, right;
  TreeNode(int val) { this.val = val; }
}

class BST {
  TreeNode root;

  // Insert
  TreeNode insert(TreeNode node, int val) {
    if (node == null) return new TreeNode(val);
    if (val < node.val)
      node.left  = insert(node.left,  val);
    else if (val > node.val)
      node.right = insert(node.right, val);
    return node;
  }

  // Search
  boolean search(TreeNode node, int val) {
    if (node == null) return false;
    if (val == node.val) return true;
    if (val < node.val)
      return search(node.left,  val);
    return search(node.right, val);
  }

  // In-order traversal (sorted order for BST)
  void inOrder(TreeNode node) {
    if (node == null) return;
    inOrder(node.left);
    System.out.print(node.val + " ");
    inOrder(node.right);
  }
}
```

Tree Traversals — All 3

```
// IN-ORDER:  Left -> Root -> Right
//           Gives sorted order for BST
void inOrder(TreeNode node) {
  if (node == null) return;
  inOrder(node.left);
  System.out.print(node.val + " ");
  inOrder(node.right);
}

// PRE-ORDER: Root -> Left -> Right
//            Good for copying tree
void preOrder(TreeNode node) {
  if (node == null) return;
  System.out.print(node.val + " ");
  preOrder(node.left);
  preOrder(node.right);
}

// POST-ORDER: Left -> Right -> Root
//             Good for deleting tree
void postOrder(TreeNode node) {
  if (node == null) return;
  postOrder(node.left);
  postOrder(node.right);
  System.out.print(node.val + " ");
}

// BFS - Level Order (uses Queue)
void levelOrder(TreeNode root) {
  if (root == null) return;
  Queue q = new LinkedList<>();
  q.offer(root);
  while (!q.isEmpty()) {
    TreeNode node = q.poll();
    System.out.print(node.val + " ");
    if (node.left  != null) q.offer(node.left);
    if (node.right != null) q.offer(node.right);
  }
}
```

Problem 10: Find Height of Binary Tree | **Difficulty:** Easy | **Tags:** Tree, Recursion

Find the maximum depth/height of a binary tree.

Java Solution:

```
int height(TreeNode root) {
  if (root == null) return 0;
  int leftH  = height(root.left);
  int rightH = height(root.right);
  return 1 + Math.max(leftH, rightH);
}
//     1
//    / \
//   2   3
//  /
// 4
// Height = 3
```

Python Solution:

```
def height(root):
  if root is None:
    return 0
  return 1 + max(height(root.left), height(root.right))
```

✓ Recursively find height of left and right subtrees. Return 1 + max of both. Base case: null node = 0.

SECTION 1F — SORTING ALGORITHMS CODE

Bubble Sort — Java

```
void bubbleSort(int[] arr) {
  int n = arr.length;
  for (int i = 0; i < n-1; i++) {
    for (int j = 0; j < n-i-1; j++) {
      if (arr[j] > arr[j+1]) {
        int temp  = arr[j];
        arr[j]    = arr[j+1];
        arr[j+1]  = temp;
      }
    }
  }
}
// O(n²) time, O(1) space
```

Merge Sort — Java

```
void mergeSort(int[] arr, int l, int r) {
  if (l >= r) return;
  int mid = (l+r)/2;
  mergeSort(arr, l,   mid);
  mergeSort(arr, mid+1, r);
  merge(arr, l, mid, r);
}

void merge(int[] arr, int l, int m, int r) {
  int n1 = m-l+1, n2 = r-m;
  int[] L = new int[n1], R = new int[n2];
  System.arraycopy(arr,l,L,0,n1);
  System.arraycopy(arr,m+1,R,0,n2);
  int i=0,j=0,k=l;
  while (i    arr[k++] = L[i]<=R[j] ? L[i++] : R[j++];
  while (i  while (j}
// O(n log n) time, O(n) space
// STABLE sort - preserves equal element order
```

Binary Search — Java

```
int binarySearch(int[] arr, int target) {
  int left = 0, right = arr.length - 1;
  while (left <= right) {
    int mid = left + (right-left)/2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) left = mid+1;
    else right = mid-1;
  }
  return -1; // not found
}
// O(log n) time, requires sorted array
```

Python — All Sorting

```
# Python built-in (Timsort O(n log n))
arr.sort()
sorted_arr = sorted(arr)

# Bubble sort
def bubble(arr):
  n = len(arr)
  for i in range(n):
    for j in range(n-i-1):
      if arr[j] > arr[j+1]:
        arr[j],arr[j+1] = arr[j+1],arr[j]

# Binary search
def binary_search(arr, target):
  l, r = 0, len(arr)-1
  while l <= r:
    mid = (l+r)//2
    if arr[mid] == target: return mid
    elif arr[mid] < target: l = mid+1
    else: r = mid-1
  return -1
```

SECTION 2 — SQL SCENARIOS (Write Queries)

These are scenario-based questions. You're given a table description and must write the query. Re-run is available — test iteratively.

SQL Problem 1: Find Top 3 Highest Paid Employees

Table: employees(id, name, department, salary). Find names and salaries of top 3 highest paid employees.

Solution:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

✓ ORDER BY salary DESC = highest first. LIMIT 3 = only top 3 results.

SQL Problem 2: Count Employees Per Department

Table: employees(id, name, department, salary). Count employees in each department, show only departments with more than 2 employees.

Solution:

```
SELECT department, COUNT(*) as emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 2
ORDER BY emp_count DESC;
```

✓ GROUP BY groups rows. COUNT(*) counts per group. HAVING filters groups (not WHERE). ORDER BY for sorted output.

SQL Problem 3: Find Employees Who Earn More Than Their Manager

Table: employees(id, name, salary, manager_id). Find employees earning more than their manager.

Solution:

```
SELECT e.name as employee, e.salary as emp_salary,
       m.name as manager, m.salary as mgr_salary
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

✓ Self-join: join table to itself. e = employee row, m = manager row. Compare salaries after join.

SQL Problem 4: Find Customers Who Never Placed an Order

Tables: customers(id, name, email), orders(id, customer_id, amount, date). Find customers with no orders.

Solution:

```
-- Method 1: LEFT JOIN
SELECT c.name, c.email
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.id IS NULL;

-- Method 2: NOT EXISTS
SELECT name, email
FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.id
);
```

✓ LEFT JOIN includes all customers. WHERE o.id IS NULL means no matching order. NOT EXISTS also works.

SQL Problem 5: Running Total / Cumulative Sum

Table: sales(date, amount). Calculate running total of sales ordered by date.

Solution:

```
SELECT date, amount,
       SUM(amount) OVER (ORDER BY date) as running_total
FROM sales
ORDER BY date;
```

✓ Window function SUM() OVER (ORDER BY date) computes cumulative sum without GROUP BY. Modern SQL feature.

SQL Problem 6: Find Duplicate Records

Table: users(id, email). Find all emails that appear more than once.

Solution:

```sql
SELECT email, COUNT(*) as count
FROM users
GROUP BY email
HAVING COUNT(*) > 1;

-- Also get the duplicate rows:
SELECT *
FROM users
WHERE email IN (
  SELECT email FROM users
  GROUP BY email
  HAVING COUNT(*) > 1
);
```

✓ HAVING COUNT(*) > 1 finds duplicated emails. Subquery gets all rows with those emails.

SQL Problem 7: Update Salary with Conditional Logic

Table: employees(id, name, department, salary). Give 10% raise to IT department, 5% raise to all others.

Solution:

```sql
UPDATE employees
SET salary = CASE
  WHEN department = 'IT' THEN salary * 1.10
  ELSE salary * 1.05
END;

-- Verify after update:
SELECT name, department, salary
FROM employees
ORDER BY department;
```

✓ CASE WHEN ... THEN ... ELSE ... END in UPDATE. Conditional logic inside SQL update statement.

SQL Problem 8: Find Second Highest Salary

Table: employees(id, name, salary). Find the second highest salary (classic interview question).

Solution:

```sql
-- Method 1: Subquery
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Method 2: DENSE_RANK
SELECT name, salary
FROM (
  SELECT name, salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) as rank
  FROM employees
) ranked
WHERE rank = 2;
```

✓ Method 1: find max excluding the overall max. Method 2: DENSE_RANK assigns rank, filter rank=2.

SECTION 3 — C++ KEY CONCEPTS

C++ OOP

C++ STL Containers

```cpp
#include
using namespace std;

class Animal {
protected:
  string name;
public:
  Animal(string n) : name(n) {}
  virtual void speak() {  // virtual = overridable
    cout << "Animal speaks" << endl;
  }
  virtual ~Animal() {}  // virtual destructor
};

class Dog : public Animal {
public:
  Dog(string n) : Animal(n) {}
  void speak() override {
    cout << name << " says Woof!" << endl;
  }
};

int main() {
  Animal* a = new Dog("Rex");
  a->speak();  // Rex says Woof! (polymorphism)
  delete a;
  return 0;
}
```

```cpp
#include
#include
#include
#include

// Vector (dynamic array)
vector v = {1,2,3};
v.push_back(4);
cout << v[0];      // 1
cout << v.size();  // 4

// Map (key-value, sorted)
map m;
m["alice"] = 95;
m["bob"]   = 87;
cout << m["alice"];  // 95

// Queue (FIFO)
queue q;
q.push(1); q.push(2);
cout << q.front();  // 1
q.pop();

// Stack (LIFO)
stack s;
s.push(10); s.push(20);
cout << s.top();  // 20
s.pop();

// Priority Queue (max-heap default)
priority_queue pq;
pq.push(3); pq.push(1); pq.push(4);
cout << pq.top();  // 4 (max)
pq.pop();
```

C++ Pointers — Quick

```cpp
int x = 10;
int* ptr = &x;  // ptr holds address of x
cout << *ptr;   // dereference = 10
*ptr = 20;      // change x via pointer
cout << x;      // 20

// References (safer than pointers)
int& ref = x;
ref = 30;   // changes x directly
cout << x;  // 30
```

C++ Key Differences from Java

```cpp
// Memory management (manual)
int* arr = new int[5];
delete[] arr;  // MUST free memory

// No garbage collector in C++
// Java has GC, C++ does not

// Multiple inheritance (C++ allows it)
class C : public A, public B {};
// Java doesn't allow multiple class inheritance

// Templates (like Java Generics)
template
T getMax(T a, T b) {
  return (a > b) ? a : b;
}
cout << getMax(3, 5);    // 5
cout << getMax(3.1,2.2); // 3.1
```

QUICK REFERENCE — COMPLEXITY CHEAT SHEET

Operation	Array	LinkedList	HashMap	BST (balanced)	Heap
Access	O(1)	O(n)	N/A	O(log n)	O(n)
Search	O(n)	O(n)	O(1) avg	O(log n)	O(n)
Insert	O(n)	O(1)	O(1) avg	O(log n)	O(log n)
Delete	O(n)	O(1)	O(1) avg	O(log n)	O(log n)

INTERVIEW TIPS — CODING SECTION

✓ Always explain your approach BEFORE coding — say 'I'll use a HashMap for O(1) lookup'

✓ Start with brute force if stuck, then optimize — shows systematic thinking

✓ Handle edge cases: empty array, null input, single element, negative numbers

✓ Write clean variable names — left/right not l/r, current not c

✓ Time complexity first, space complexity second — always state both

✓ For SQL: start simple (SELECT * WHERE), add complexity step by step, use re-run to verify

✓ LinkedList problems: always draw the pointer changes on paper first

✓ Recursion: always define base case first, then recursive case

✓ Priority Queue = heap. Max-heap: largest out first. Min-heap: smallest out first.

✓ Elevator going up/down with floors = Priority Queue where floor number is priority